Queuing System Simulation

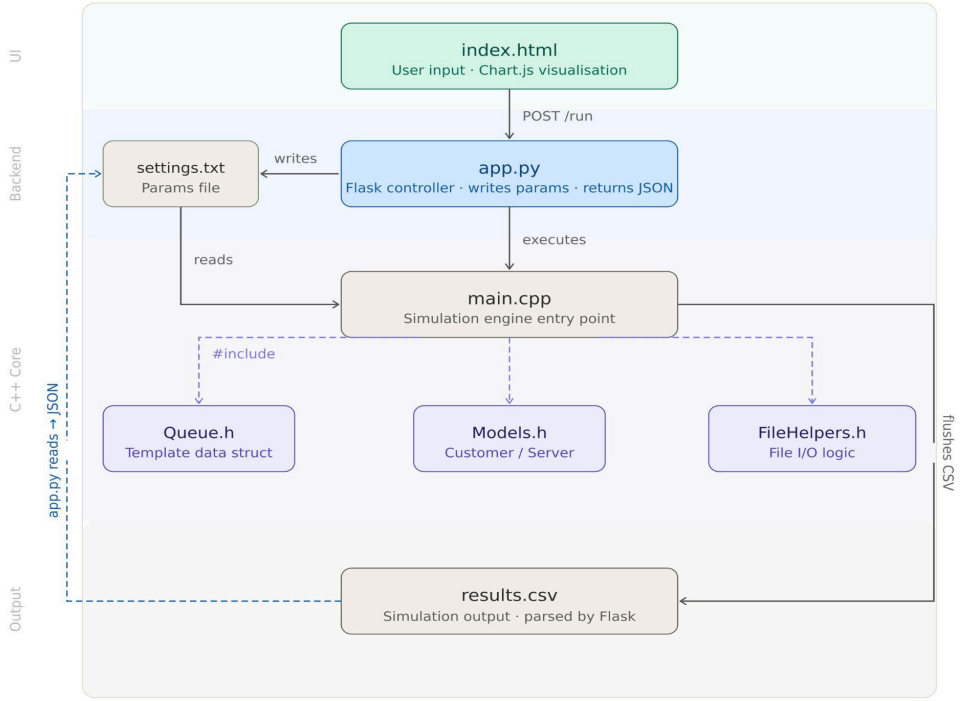
Team Members:

- Ziad Waleed Sallam (Queue Data Structure)
- Abdulrahman Magdy Saad (Object Models)
- Nour eldin Majd Salem (Core Simulation Engine)
- Mahmoud Mostafa (File I/O & Metrics)
- Amr Hany (UI, & Data Visualization)

Introduction

This project goal was designing and implementing a simulation of a multi-server customer service queue. The core application is the practical implementation of a First-In-First-Out (FIFO) **Queue** implemented from scratch in C++. By simulating random customer arrivals and service durations, the system evaluates key performance metrics such as average wait times, maximum queue lengths, and server utilization over a period of time

Queue dependency map



---

1. Core Data Structure: The Custom Queue (Built by Ziad)

Ziad implemented a custom, highly optimized Queue data structure from scratch (Queue.h) instead of using STL such as (<queue>)

Architecture:

The queue is implemented as a **Singly Linked List** using C++ Templates (template <typename T>). which allows the queue to accept any data type from the Customer class dynamically upon request.

Key DSA Complexities & Design Choices:

1. **Node{}** which is the core for our Linked list: makes use of pointer to traverse the list through **Next = nullptr**
2. **Front()**: points to the data head without deleting it so it can be dequeue (served).
3. **Rear()**:Points to the very last person who just joined.
4. **enqueue(T x):** A new node is created on the Heap using new Node<T>(x). If queue is not empty, the Rear->next is set to this new node.If the queue is empty, the Front is set to this new node (since the first person is both the front and the back) we update the Rear pointer to be moved to this new node
5. **dequeue()** [Time Complexity: O(1)]**:** Elements are removed using the Front pointer but it first checks `empty()` to prevent any segmentation fault then the memory is safely released using delete Front to prevent any memory leaks during long simulations.
6. **T front() const :** This returns the data of the first node without deleting it, If the queue is empty it returns a "default" T( ) this is to prevent the main.cpp from crashing if it asks for a customer when the queue is empty.
7. **Queue():** Default constructor it sets the Front = nullptr, Rear = nullptr, Size=0,this prevents the possibility of any garbage value win calling the empty() or enqueue() during the program
8. **~Queue():** the destructor is built to fulfill one task cleaning any node created on the Heap, by calling dequeue() and using delete it releases Front node from memory safely
9. **Queue(const Queue &x):** this deep copy constructor prevents any shallow copies where we only point to the original memory address instead of iterating through the original queue and allocation new nodes in the new queue.
10. **(operator=):** This method handles the transfer of data between two existing queues. It includes the assignment operator (if (this == &x)) to prevent a queue from accidentally deleting its own data during the assignment operation.

Time complexity:

Operation	Complexity
Enqueue	O(1)
Dequeue	O(1)
Empty/Size	O(1)
Size	O(n)

2. Object Modeling & Encapsulation (Built by Abdulrahman)

The Customer class:
Customer class is a dynamic unit that carries all the data point from the arrival generator to the Queue, and finally to a Server.
Logic and Attributes:

1. Id: Tracks unique identifier for the arrival moment,
2. arrivalTimes: records the moment of entry

3. requiredServiceTime: a random value that tells the sim how many ticks() will the server be occupied by a specific customer
4. waitTime: is calculated using (waitTime = currentTime - arrivalTime) and is used for the end of queue analysis

Implementation:

1. **Encapsulation:** all members are private and access is only provided through getters such as getRequiredServiceTime() const to prevent any accidental modification to the customers
2. **Constructors:** includes both a default constructor Customer() for buffer initializations and a parameterized constructor for the in-real-time initialization during the simulation.

The Server Class:
It models the service points for our simulations
Core Methods:

1. **Tick():** gets called by the engine at the end of each minute to decrement remainingServiceTime() if the timer is ever at 0 it toggles busy to become false, so it can take a new customer
2. **isBusy():** a boolean that is used to check if a server is available for a dequeued customers

Memory Optimization:

1. **new Server[numServers]:** using a dynamic array for the servers allows for a flexible system to test all the possible outputs

4. Core Simulation Engine: Implementation & Logic (Built by Nour)

Architectural Overview

The simulation is designed as a Discrete-Time Event Simulation, our engine progresses increments (minutes) with each tick() of the clock, the engine performs four critical tasks

Execution Flow & Logic Gates

1. **Initialization:** At the start of execution, the engine calls readSettings() from the FileHelpers module, where external parameters (Servers, Arrival Probability, etc.) are injected into the engine at runtime rather than being hard-coded.
2. **Dynamic Memory**: allocating a dynamic array of servers: Server* servers = new Server[numServers], This allows for flexibility on user input.

The Main Simulation Loop

The engine uses a loop that terminates once currentTime reaches maxSimulationTime.

1. **Customer Generation:** Using a random number generator (rand() % 100 + 1), the engine determines arrival percentage based on the arrivalProbability, then a new Customer() object is instantiated and pushed into Ziad's Linked-List Queue**.**
2. **Resource Update (The Server Tick):**
   iterating through the array of servers and calls servers[i].tick(). This decrements the remaining service time for any busy server, simulating time passing.
3. **Queue Processing (The Dequeue Logic):**
   every server is checked, If a server is not busy and the queue is not empty, the engine:
   - Dequeues the front customer.
   - Calculates the wait time: (waitTime = currentTime - arrivalTime).
   - Updates global statistics (Total wait time, customers served).
   - Assigns the customer to the free server.

Real-time Metric Capture

1. At the end of every minute, the engine captures a MinuteSnapshot. This records the current state of the queue and the cumulative averages. These snapshots are stored in a std::vector<MinuteSnapshot>, which acts as a high-speed memory buffer before being written to the disk.

Time complexity:

Operation	Complexity
Arrival Check	O(1)

Server Ticking	O(n)
Queue Dispatching	O(n)
Total Per Minute	O(n)
Total Simulation	O(m×n)

5. Data Persistence & File I/O(Built by Mahmoud)

The Files-Helper is the bridge between the C++ simulation and the final data visualization. It is designed to handle configuration inputs and performance logging without interfering with the speed of the core algorithm.

Input Management & Fallbacks

The system adds values by utilizing an external configuration file.

1. **Dynamic Loading:** At startup, the engine reads key parameters—such as the number of servers and arrival probabilities from a settings file.
2. **Error handling:** An Error mechanism is implemented so that if the configuration file is missing, the simulation automatically adopts safe default values. This ensures the program is always operational.

Efficient Performance Logging

1. **System analyzation**: the queue is recording the behavior of the simulation at every virtual minute.
2. **High-Speed Buffering:** Rather than writing to the disk every minute the system writes in a high-speed memory buffer.
3. **Data Captured:** Each recording of the current queue length, the total number of customers served, and the cumulative average wait time.

Output (CSV)

1. **Exchange:** CSV file was chosen as it allows the data generated by C++ to be read by other platforms, specifically the Web UI's Python backend.
2. **Data Integrity:** ensuring the final charts and graphs are exactly representative of the Queue's performance throughout the simulation.

6. User Interface & Data Visualization (Built by Amr)

The Control Unit for the simulation is through the UI. It changes a CLI C++ program to a full-stack web app to real-time Adjustments and visual analysis after every simulation run.

Architecture

using a **Bridge** that connects three app layers:

1. **Backend (C++):** The high-performance simulation engine.
2. **Server (Python/Flask):** The coordinator that manages files and processes.
3. **Frontend (HTML5/JavaScript):** The interactive dashboard.

The Python (app.py)

The Flask server has the role of a mediator between C++ and the frontend HTML, such as managing web requests and executing commands.

1. **Process Management**: Using the subprocess module, Python asks the compiled C++.exe then waits until the simulation finishes before moving to data parsing.
2. **The Settings file:** Python gets inputs from (Servers, Arrival Rate and outputs into settings.txt. This is the primary way the UI communicates with the C++.
3. **JSON:** Once the C++ creates the results.csv, the Python backend reads the file and converts it to JSON. Translating a flat file into structured objects that the frontend can map.

The Dashboard (index.html)

The UI was designed to be "Data-First," focusing on the metrics generated by the queue.

1. **Asynchronous (AJAX):** The UI uses fetch() requests to the Python server. This allows running a simulation and seeing the results update instantly without the webpage refreshing.
2. **Visualization (Chart.js):**
    - **Queue Length Chart:** Visually representing the Bottleneck the queue_length for every minute of the simulation.
    - **Wait Time Chart:** A graph showing avg_wait_time. allowing to see how wait times compound as the queue grows.
3. **Responsive Styling:** Built using CSS which keeps the data as the focus point of the UX

Operational Workflow

1. **Input:** User sets the "Arrival Probability" and "Number of Servers" on the dashboard.
2. **Trigger:** Flask receives the request and updates settings.txt.
3. **Execution:** The C++ runs, utilizing Ziad's **Queue** and Nour's **Logic** processing the data.
4. **Parsing:** Python reads the results.csv O/P.

5. **Output:** JavaScript renders the final graphs, showing exactly what happened through the simulation.

Complexity

Component	Performance Note	Responsibility
Flask (Python)	Minimal overhead; acts only as a trigger.	Process Coordination
Subprocess	Runs at native speed ($O(m \times n)$).	C++ Execution
Chart.js	$O(n)$, n is the number of minutes simulated.	Data Rendering